Smalltalk Interpreter

Smalltalk (ST) is a pure object oriented programming language. In ST, **all values** are considered objects. The interpreter that we write is for is a small approximate subset of ST (See the grammar for it below).

Unlike mini lisp which had only one value (integer), our ST has three type of values (all are objects). They are **int**, **sym** (strings), and **object instance**.

In minilisp our eval only had to pass back an integer. Here we have to package our ST values in the following way:

```
struct  STVALUEREC
{
      CLASS owner;  //every value is owned by some one
      STVALUETYPE vtype;// INT,  SYM,  USER
      int intval;  //if its INT type
      NAME symval; //if its SYM type
      ENV userval; //if its USER instance
};
```

Hence the EXPREC has to be modified:

```
struct EXPREC
{
      EXPTYPE etype; //what type of expression
      STVALUE valu; // This is the change from mini lisp
      NAME varble;
      NAME optr;
      EXPLIST args;
};
```

PART ONE DUE Nov 9

Do the following:

a) Modify the symbol table.

b) Create a record to store small talk values. Create a linked list to store class records (come to class for details).

c) Write a function ParseClass() that will parse a class, create a class record and link it in the linked list of class records.

PART TWO DUE NOV 30

Finish the interpreter by modifying eval and a few other functions (not much), please pay attention in class.

The Mini Smalltalk Interpreter

The interpreter is interactive. The user enters three kinds of inputs

- **Function definitions (similar to mini lisp):**

```
(define double (x) (+ x x) )
```

- **Expressions (similar to minilisp):**

```
(double 10)
```

- **Class definitions such as (addition to minilisp):**

```
(class FinancialHistory Object
    (cashOnHand incomes expenditures)
    (define initFinancialHistory (amount)
         (begin
              (set cashOnHand amount)
              (set incomes (mkDictionary))
              (set expenditures (mkDictionary))
             self))
    (define receive:from: (amount source)
         (begin
            (at:put: incomes source (+
(totalReceivedFrom: self source) amount))
             (set cashOnHand (+ cashOnHand amount))))
    (define spend:for: (amount reason)
         (begin
           (at:put: expenditures reason (+
(totalSpentFor: self reason) amount))
            (set cashOnHand (- cashOnHand amount))))
    (define cashOnHand () cashOnHand)
    (define totalReceivedFrom: (source)
         (if (includesKey: incomes source)
              (at: incomes source)
              0))
    (define totalSpentFor: (reason)
         (if (includesKey: expenditures reason)
              (at: expenditures reason)
              0))
)
```

- Function and class definitions are simply remembered by the interpreter.

- Expressions are evaluated. Evaluating an expression is the same as running a program in most other languages.

Note that a class definition has methods, these belong to the class that defines the method. However, the syntax for the method is exactly the same as for a function. Hence, we can make use of our previous "ParseDef" to parse a method with some very simple modifications.

Syntax

```
input --> expression | fundef | classdef

classdef --> (class class class inst-vars methoddef+)

inst-vars --> (variable*)

methoddef --> fundef

fundef --> (define function arglist expression)
                                        ;same as lisp
  arglist--> (variable*)

expression --> value | variable
               | (if expression expression expression )
               | (while expression)
               | (set variable expression)
               | (begin expression+)
               | (optr expression*)

optr --> function | value-op

value --> integer | #symbol

value-op --> new | + | - | * | / | = | < | > | print

function --> name

variable --> name

class --> name
```

integer--> sequence of digits, possibly preceded by a minus sign

name --> any sequence of characters not an integer, and not containing (,), ;, or space

A function cannot be one of the keywords define, if, while, begin or set or any of the value-ops. But methods can be overloaded with value ops.

Comments are introduced by the character ';'and continue to the end of the line.

A session is terminated by entering quit.

Semantics

symbols: Can be either integers, symbols, or class instances (objects). The symbol #nil is used to represent "bad" or "undefined" values.

self: The definition of a method always has one implicit argument, which can be referred to by the name self.

new: This function creates a new object of class C when applied to the variable called C, initializing its instance variables to the integers zero.

Redefinition of value-op's: A small extension not previously noted is that value-ops are allowed to be used as messages.

Inheritance: The instance variables representing an object of class C are those named in C or in any of its ancestors. Object of a class C can respond to messages defined in as ancestor of C.

Method search: when a message **m** is sent to an object of class C, the interpreter first looks in C for a method **m**, if it fails, it looks in the supper class and so on.

The class Object: We start with one class called **Object**, which has instance variable **self** and as its objects all the **integers** and **symbols**. All other classes inherit the instance variable self, which is initialized whenever a new object is created.

Scope rules: For the variables is static and for functions it is dynamic.

Functions and messages: There are two types of functions calls, global and message sends. First check to make sure it's a message if not it must be a global function else error.

Working Example:

```
(define mkFinancialHistory (amount)
     (initFinancialHistory (new FinancialHistory)
amount))

(set myaccount (mkFinancialHistory 1000))
(spend:for: myaccount 50 #insurance)
(receive:from: myaccount 200 #salary)
(cashOnHand myaccount)
1150
(spend:for: myaccount 100 #books)
(cashOnHand myaccount)
1050
(set myaccount (mkDeductibleHistory 1000))
(spend:for: myaccount 50 #insurance)
(receive:from: myaccount 200 #salary)
(cashOnHand myaccount)
1150
(totalDeductions myaccount)
0
```

```
(spend:Deduct: myaccount 100 #mortgage)
(cashOnHand myaccount)
1050
(totalDeductions myaccount)
100
(spend:for:deduct: myaccount 100 #3-martini-lunch 50)
(cashOnHand myaccount)
950
(totalDeductions myaccount)
150
```